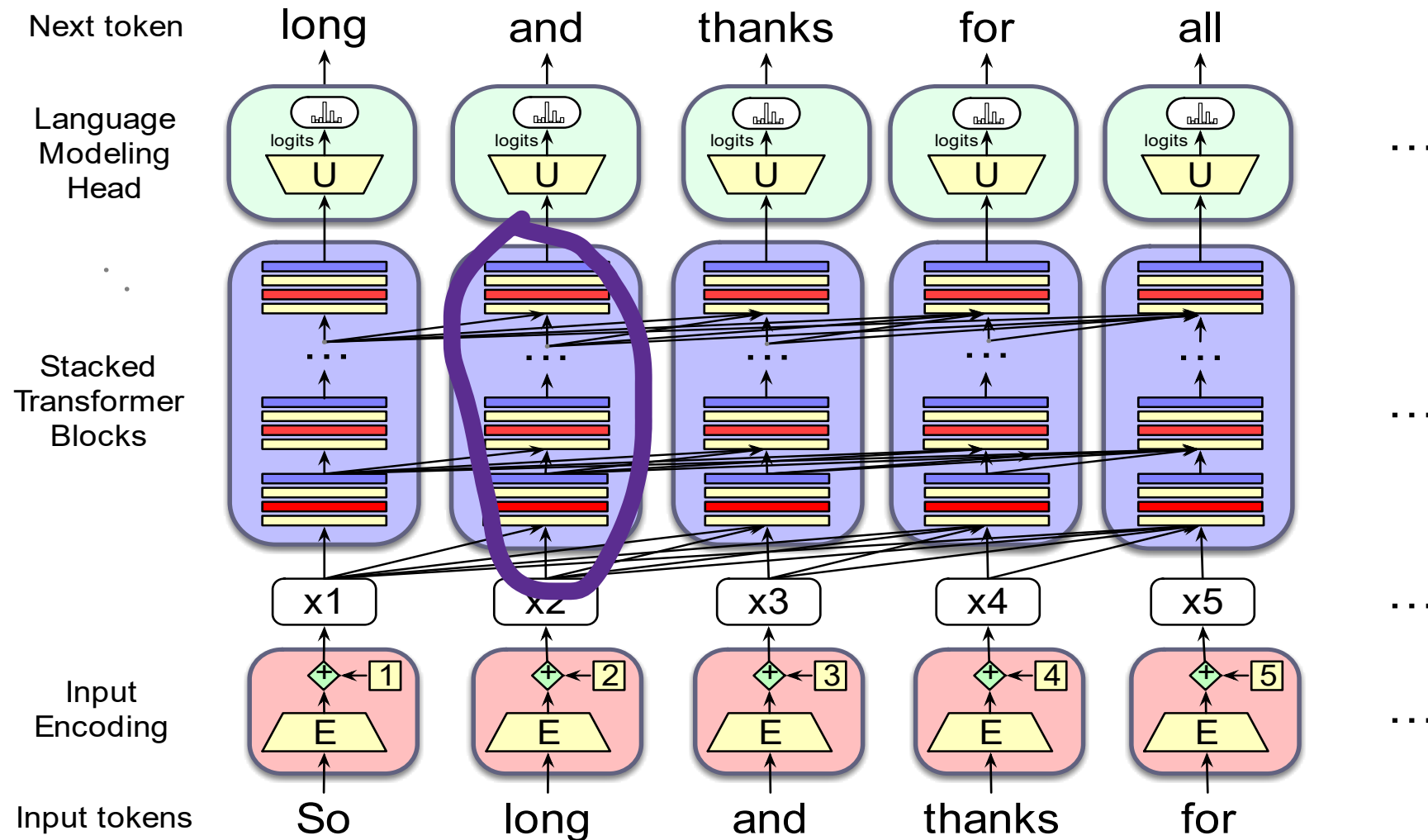


Transformers

Introduction to Transformers

Reminder: transformer language model



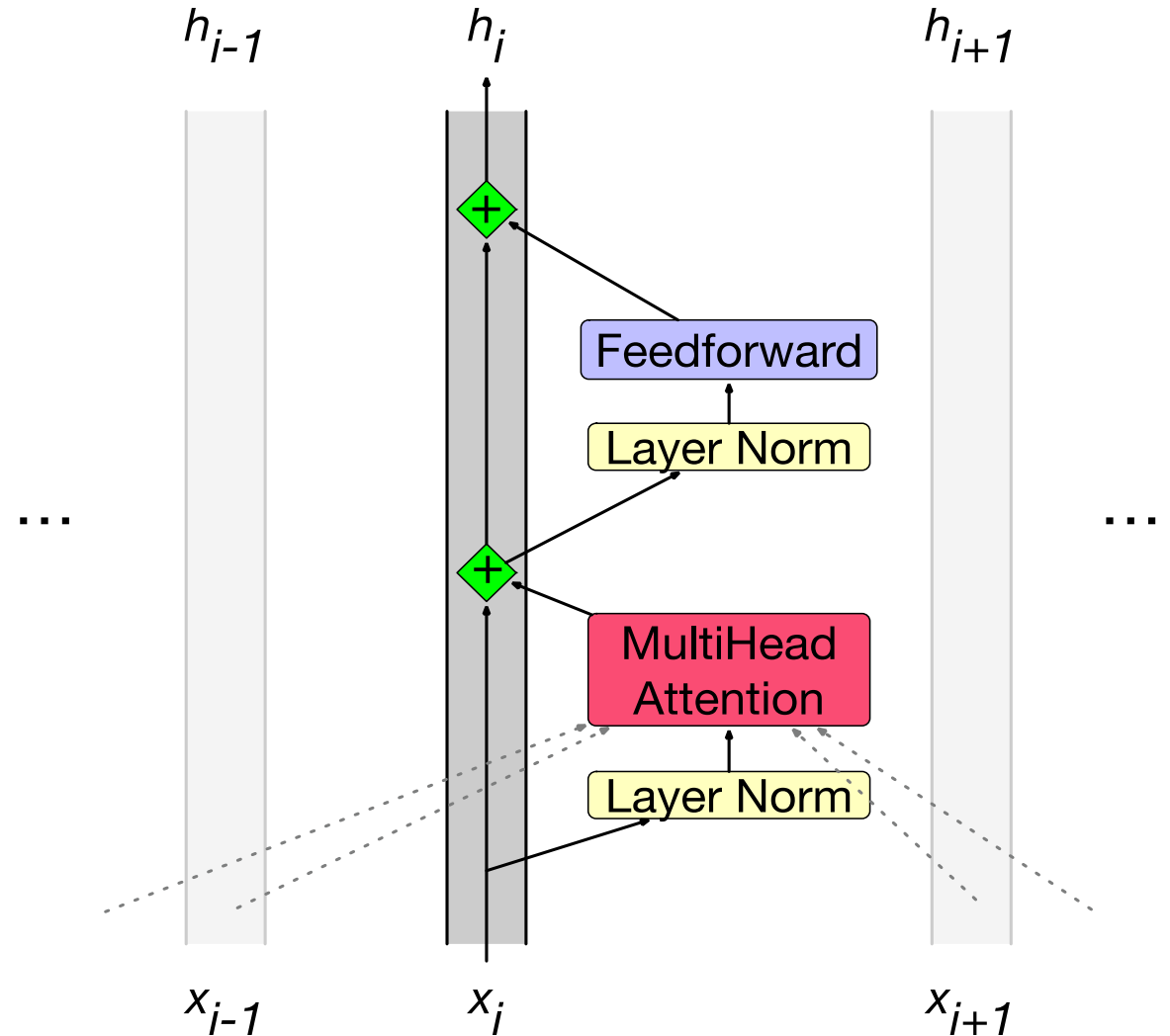
Single Transformer Block

The residual stream: each token gets passed up and modified

We'll need nonlinearities, so a feedforward layer

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i \mathbf{W}_1 + b_1) \mathbf{W}_2 + b_2$$

Layer norm: the vector $\mathbf{x}_i$ is normalized twice



Layer Norm

Layer norm is a variation of the z-score from statistics, applied to a single vector in a hidden layer

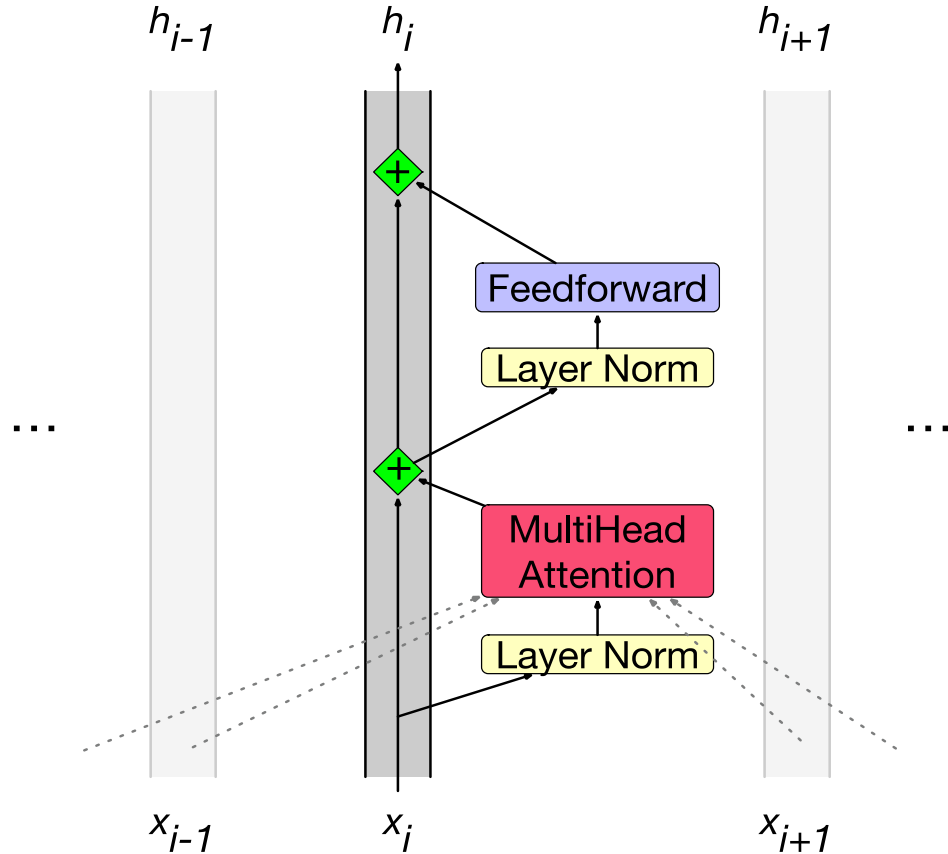
$$\mu = \frac{1}{d} \sum_{i=1}^d x_i$$

$$\sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

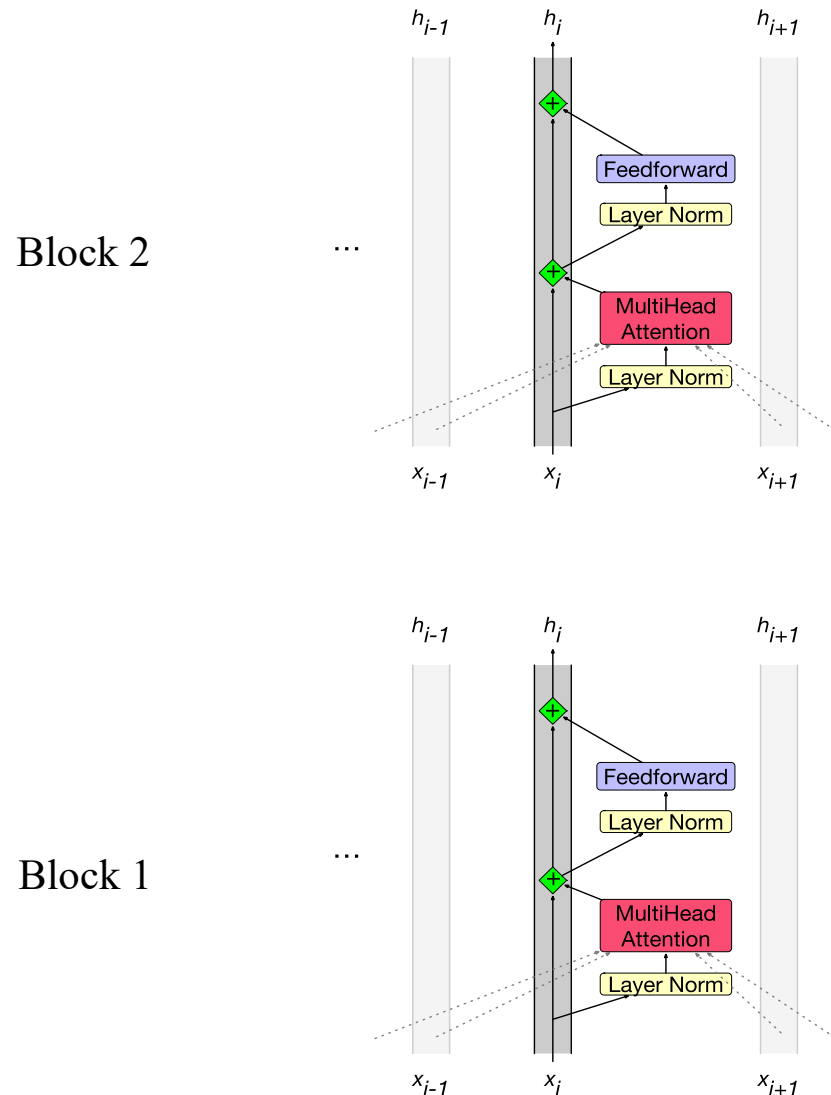
$$\text{LayerNorm}(\mathbf{x}) = \gamma \frac{(\mathbf{x} - \mu)}{\sigma} + \beta$$

Putting together a single transformer block



$$\begin{aligned}\mathbf{t}_i^1 &= \text{LayerNorm}(\mathbf{x}_i) \\ \mathbf{t}_i^2 &= \text{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{x}_1^1, \dots, \mathbf{x}_N^1]) \\ \mathbf{t}_i^3 &= \mathbf{t}_i^2 + \mathbf{x}_i \\ \mathbf{t}_i^4 &= \text{LayerNorm}(\mathbf{t}_i^3) \\ \mathbf{t}_i^5 &= \text{FFN}(\mathbf{t}_i^4) \\ \mathbf{h}_i &= \mathbf{t}_i^5 + \mathbf{t}_i^3\end{aligned}$$

A transformer is a stack of these blocks
so all the vectors are of the same dimensionality d



Each token vector X_i at the input to the block has dimensionality d , and the output h_i also has dimensionality d .

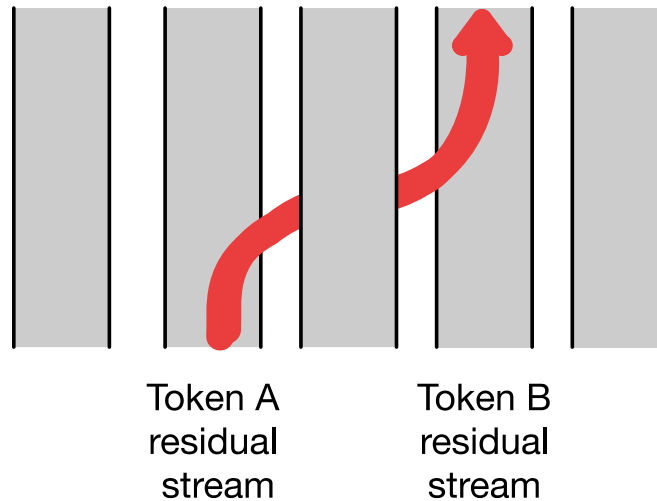
Transformers for large language models stack many of these blocks, from **12 layers** (used for the T5 or GPT-3-small language models) to **96 layers** (used for GPT-3 large), to even more for more recent models.

Residual streams and attention

Notice that all parts of the transformer block apply to 1 residual stream (1 token).

Except attention, which takes information from other tokens

[Elhage et al. \(2021\)](#) show that we can view attention heads as literally moving information from the residual stream of a neighboring token into the current stream .



Parallelizing computation using $\mathbf{X}$

For attention/transformer block we've been computing a **single** output at a **single** time step i in a **single** residual stream.

But we can pack the N tokens of the input sequence into a single matrix $\mathbf{X}$ of size $[N \times d]$.

Each row of $\mathbf{X}$ is the embedding of one token of the input.

$\mathbf{X}$ can have 1K-32K rows, each of the dimensionality of the embedding d (the **model dimension**)

$$\mathbf{Q} = \mathbf{XW}^{\mathbf{Q}}; \quad \mathbf{K} = \mathbf{XW}^{\mathbf{K}}; \quad \mathbf{V} = \mathbf{XW}^{\mathbf{V}}$$

$$QK^T$$

Now can do a single matrix multiply to combine Q and K^T

N	q1•k1	q1•k2	q1•k3	q1•k4
	q2•k1	q2•k2	q2•k3	q2•k4
	q3•k1	q3•k2	q3•k3	q3•k4
	q4•k1	q4•k2	q4•k3	q4•k4
	N			

Parallelizing attention

- Scale the scores, take the softmax, and then multiply the result by V resulting in a matrix of shape $N \times d$
 - An attention vector for each input token

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

Masking out the future

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

- What is this mask function?
 $\mathbf{QK}^\top$ has a score for each query dot every key, *including those that follow the query.*
- Guessing the next word is pretty simple if you already know it!

Masking out the future

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

Add $-\infty$ to cells in upper triangle
The softmax will turn it to 0

N

q1•k1	$-\infty$	$-\infty$	$-\infty$
q2•k1	q2•k2	$-\infty$	$-\infty$
q3•k1	q3•k2	q3•k3	$-\infty$
q4•k1	q4•k2	q4•k3	q4•k4

N

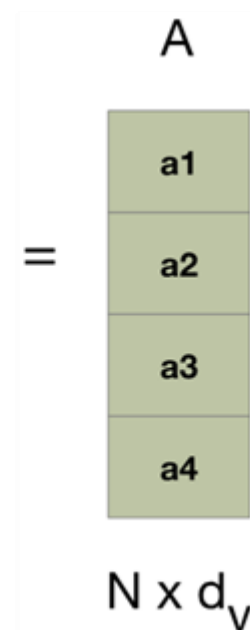
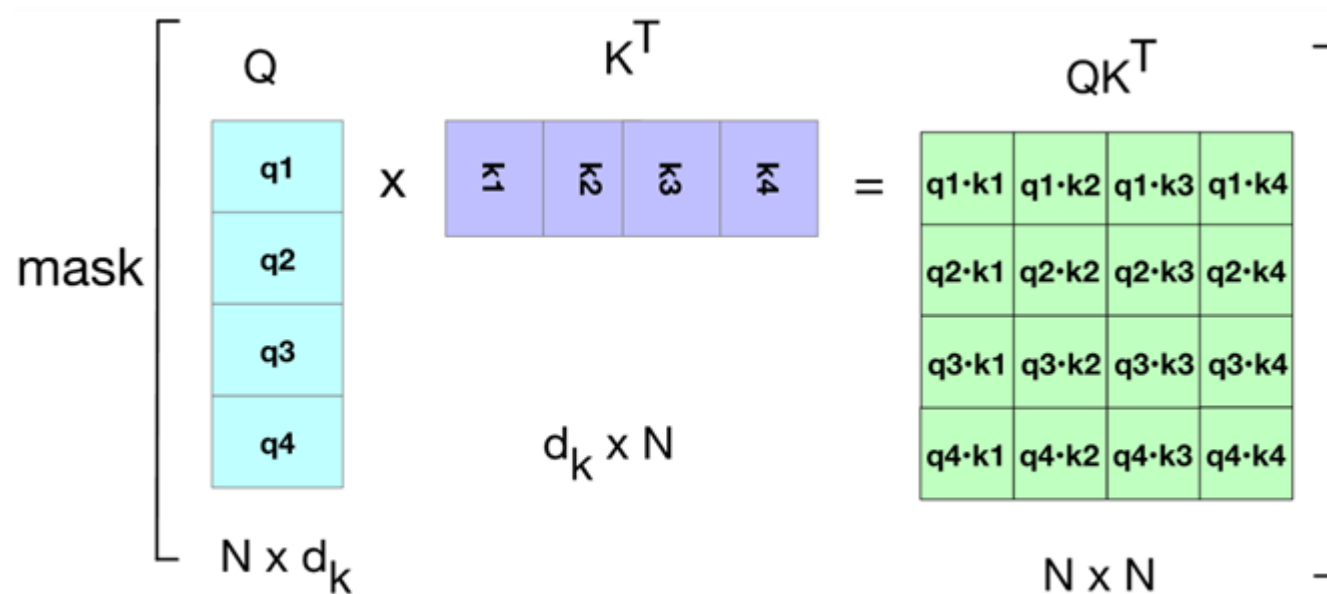
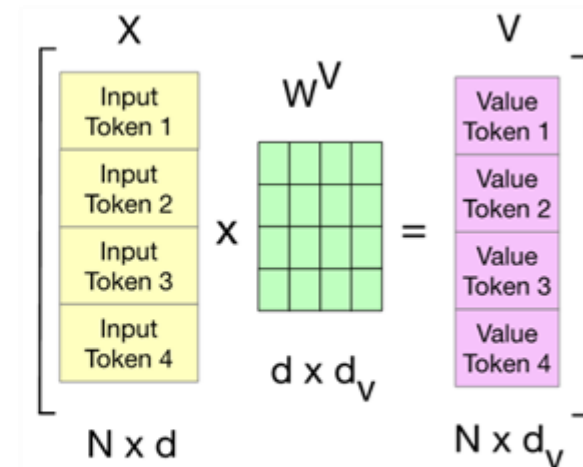
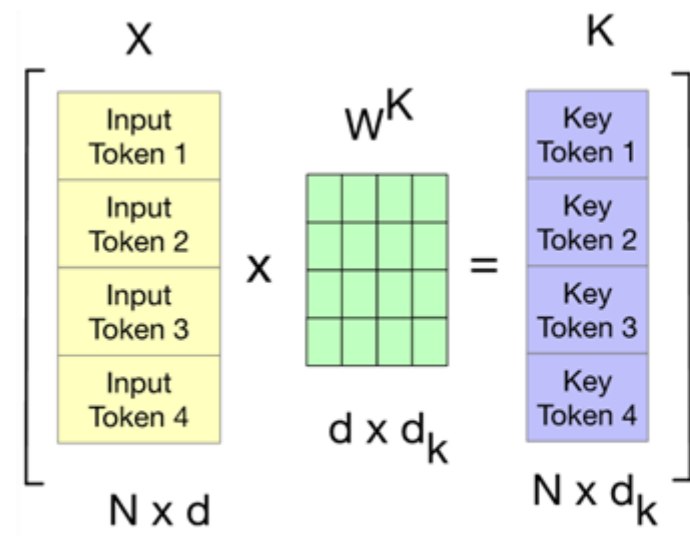
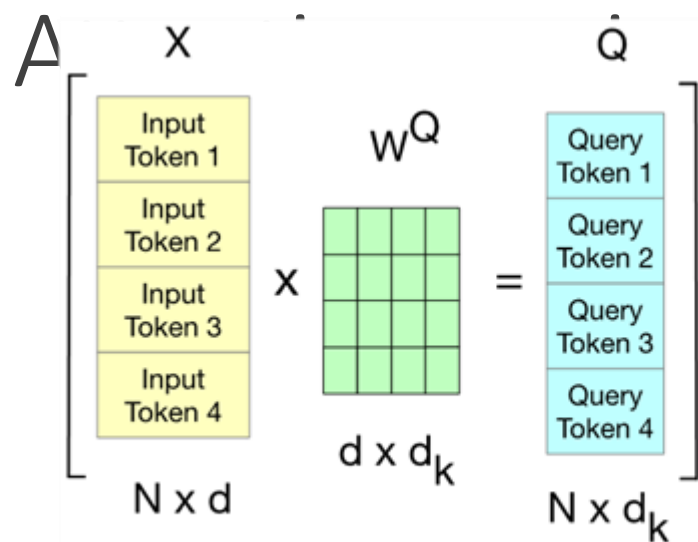
Another point: Attention is quadratic in length

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

N

q1•k1	−∞	−∞	−∞
q2•k1	q2•k2	−∞	−∞
q3•k1	q3•k2	q3•k3	−∞
q4•k1	q4•k2	q4•k3	q4•k4

N



Attention again

QK^T masked

$q1 \cdot k1$	$-\infty$	$-\infty$	$-\infty$
$q2 \cdot k1$	$q2 \cdot k2$	$-\infty$	$-\infty$
$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$-\infty$
$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$

$N \times N$

$$\begin{matrix} & V \\ X & \begin{bmatrix} v1 \\ v2 \\ v3 \\ v4 \end{bmatrix} \\ & N \times d_v \end{matrix} = \begin{matrix} & A \\ & \begin{bmatrix} a1 \\ a2 \\ a3 \\ a4 \end{bmatrix} \\ & N \times d_v \end{matrix}$$

Parallelizing Multi-head Attention

$$\mathbf{Q}^i = \mathbf{XW}^{\mathbf{Q}^i}; \quad \mathbf{K}^i = \mathbf{XW}^{\mathbf{K}^i}; \quad \mathbf{V}^i = \mathbf{XW}^{\mathbf{V}^i}$$

$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q}^i, \mathbf{K}^i, \mathbf{V}^i) = \text{softmax} \left(\frac{\mathbf{Q}^i \mathbf{K}^{i\top}}{\sqrt{d_k}} \right) \mathbf{V}^i$$

$$\text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \dots \oplus \mathbf{head}_h) \mathbf{W}^0$$

Parallelizing Multi-head Attention

$$\mathbf{O} = \text{LayerNorm}(\mathbf{X} + \text{MultiHeadAttention}(\mathbf{X}))$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{O} + \text{FFN}(\mathbf{O}))$$

or

$$\mathbf{T}^1 = \text{MultiHeadAttention}(\mathbf{X})$$

$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1$$

$$\mathbf{T}^3 = \text{LayerNorm}(\mathbf{T}^2)$$

$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3)$$

$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5)$$

Token and Position Embeddings

The matrix X (of shape $[N \times d]$) has an embedding for each word in the context.

This embedding is created by adding two distinct embeddings for each input

- token embedding
- positional embedding

Token Embeddings

Embedding matrix E has shape $[|V| \times d]$.

- One row for each of the $|V|$ tokens in the vocabulary.
- Each word is a row vector of d dimensions

Given: string "*Thanks for all the*"

1. Tokenize with BPE and convert into vocab indices

$w = [5, 4000, 10532, 2224]$

2. Select the corresponding rows from E , each row an embedding

- (row 5, row 4000, row 10532, row 2224).

Position Embeddings

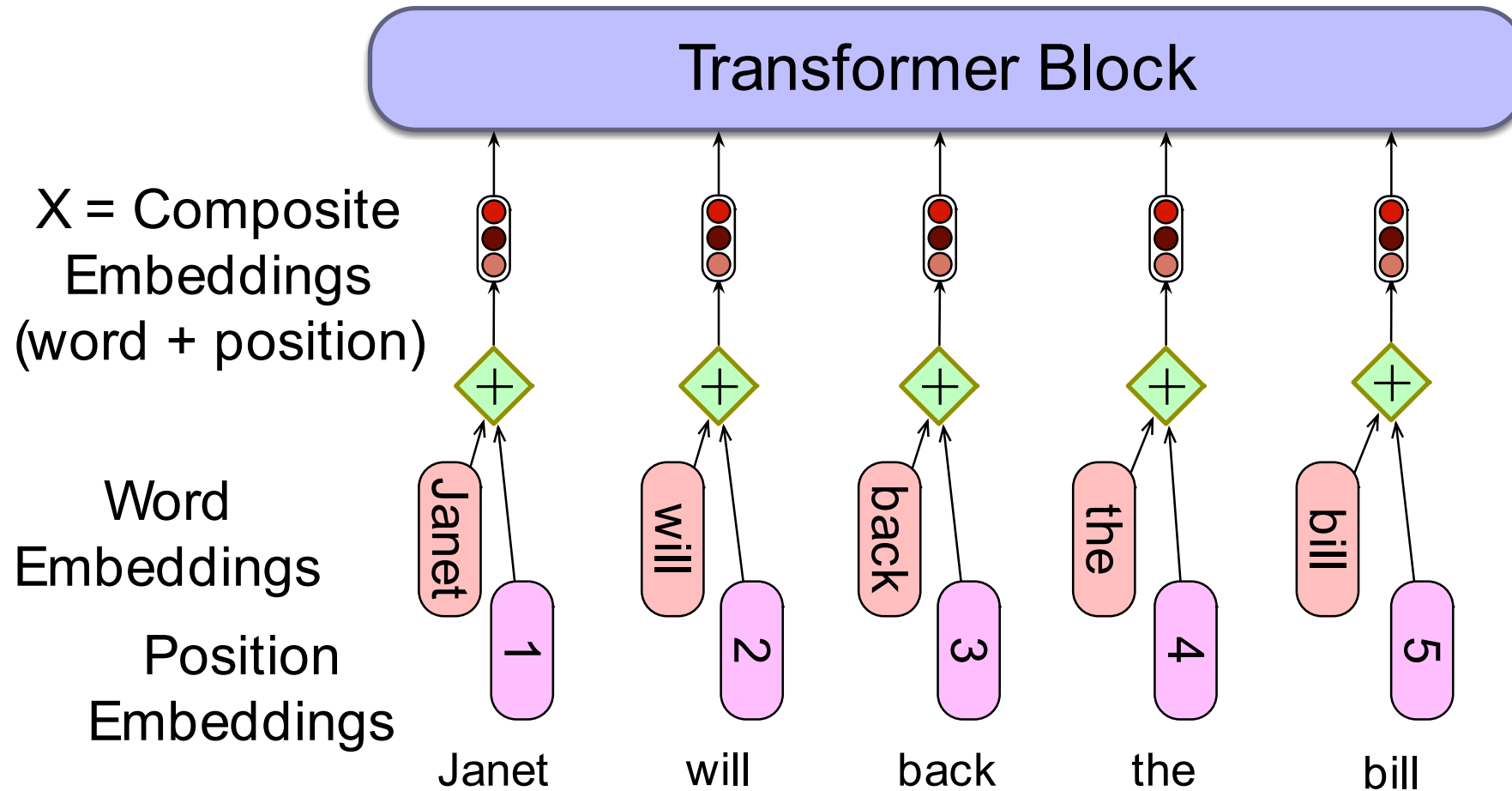
There are many methods, but we'll just describe the simplest: absolute position.

Goal: learn a position embedding matrix E_{pos} of shape $[1 \times N]$.

Start with randomly initialized embeddings

- one for each integer up to some maximum length.
- i.e., just as we have an embedding for token *fish*, we'll have an embedding for position 3 and position 17.
- As with word embeddings, these position embeddings are learned along with other parameters during training.

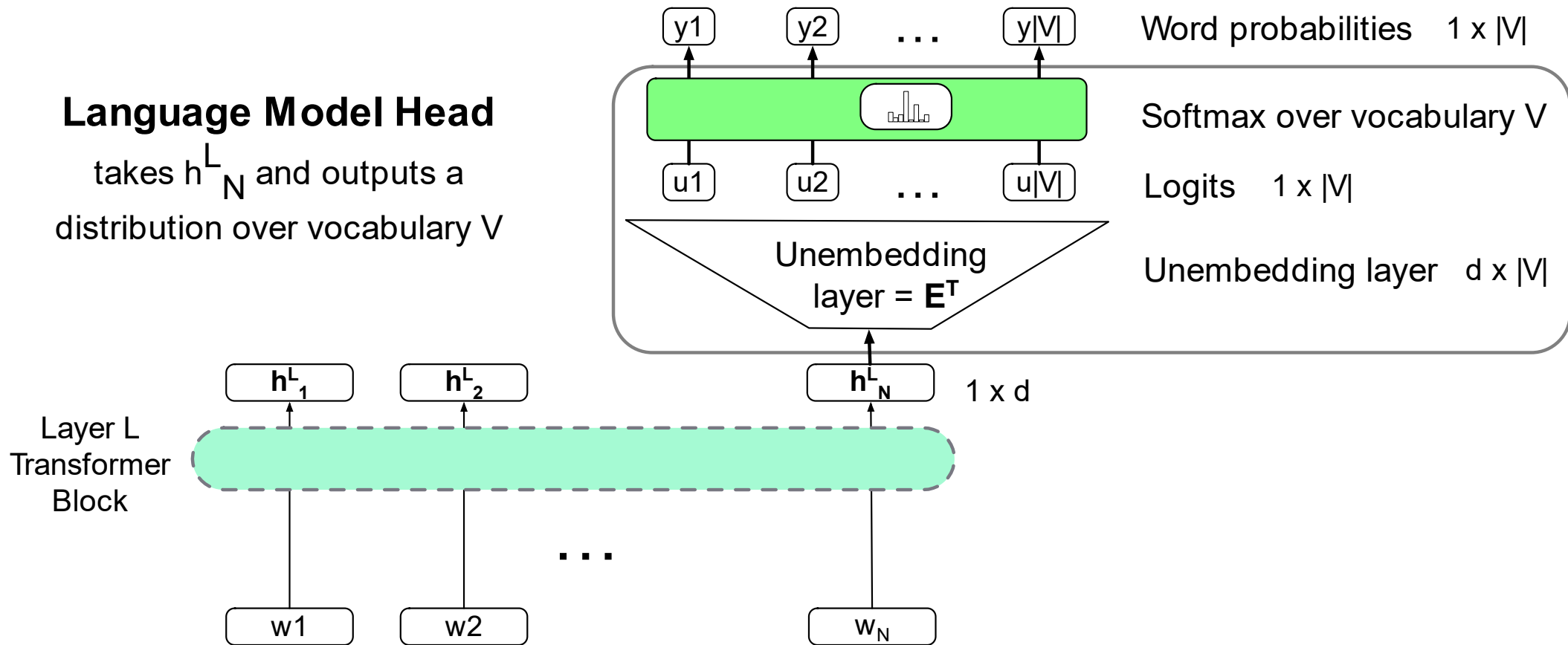
Each x is just the sum of word and position embeddings



Language modeling head

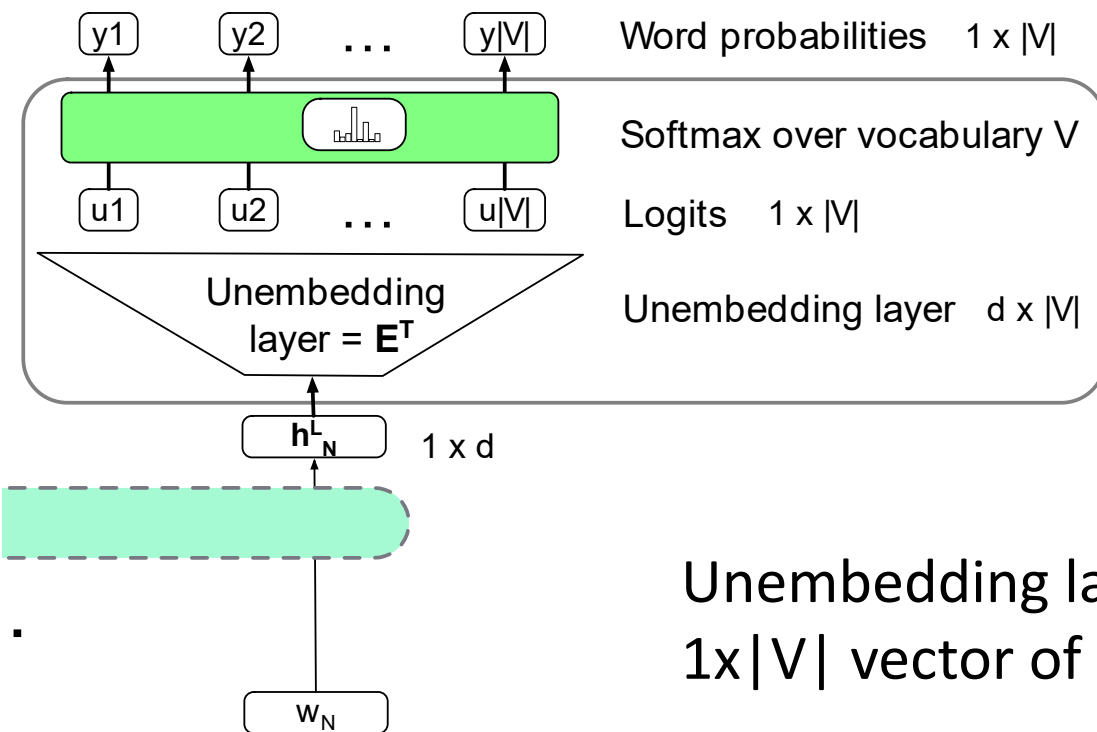
Language Model Head

takes h_N^L and outputs a distribution over vocabulary V



Language modeling head

Unembedding layer: linear layer projects from h_N^L (shape $[1 \times d]$) to logit vector



Why "unembedding"? **Tied** to E^T

Weight tying, we use the same weights for two different matrices

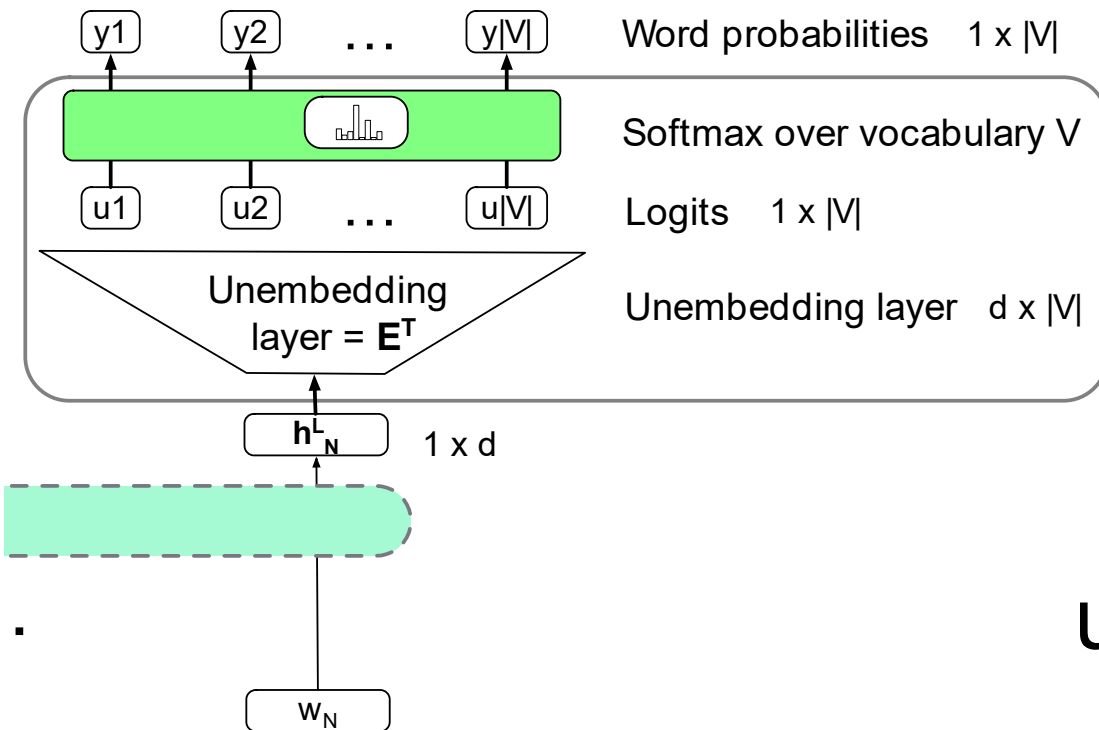
Unembedding layer maps from an embedding to a $1 \times |V|$ vector of logits

Language modeling head

Logits, the score vector u

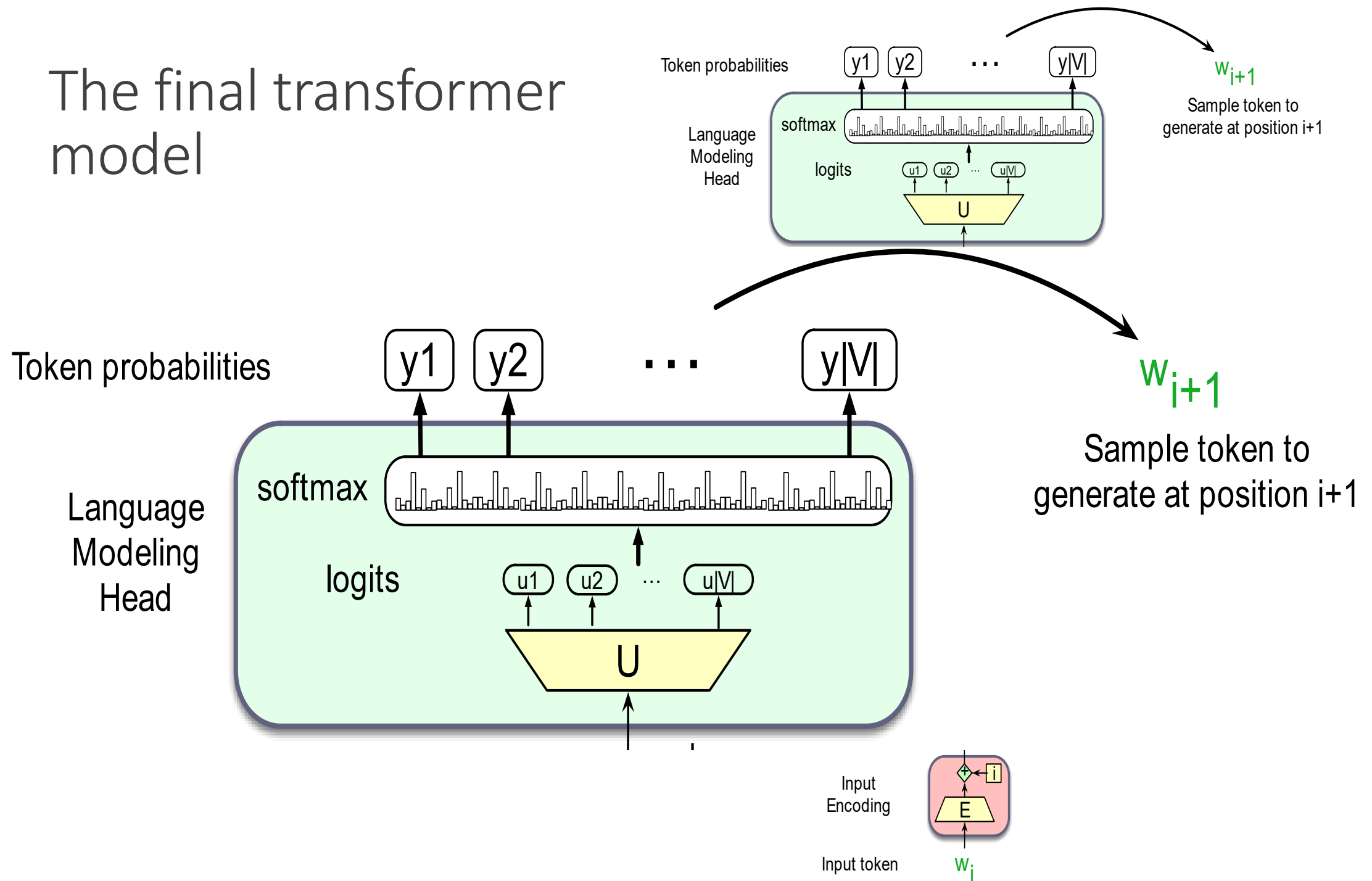
One score for each of the $|V|$ possible words in the vocabulary V .
Shape $1 \times |V|$.

Softmax turns the logits into probabilities over vocabulary.
Shape $1 \times |V|$.



$$u = h_N^L E^T$$
$$y = \text{softmax}(u)$$

The final transformer model



References

- <https://www.youtube.com/watch?v=KMHkbXzHn7s>
- <https://www.youtube.com/watch?v=bCz4OMemCcA&t=1243s>
- <https://www.youtube.com/watch?v=ZhAz268Hdpw&t=1439s>